O'REILLY®
Technical Guide

The LLM Mesh

A Practical Guide to Using Generative AI in the Enterprise

Early Release
RAW & UNEDITED

Compliments of

data iku

Kurt Muehmel

Dataiku

data iku

Efficient and Governed Generative AI With Dataiku

THE LLM MESH

A common backbone for GenAI applications, enabling choice and flexibility among the growing number of models and providers.

AI-POWERED ASSISTANTS

Go faster and farther with AI Prepare, AI Code Assistant, and AI Explain, all of which improve efficiency and the overall product.

DATAIKU ANSWERS

A packaged, scalable web application to democratize enterprise-ready LLM chat and retrieval-augmented generation (RAG).

LLM-POWERED DATA

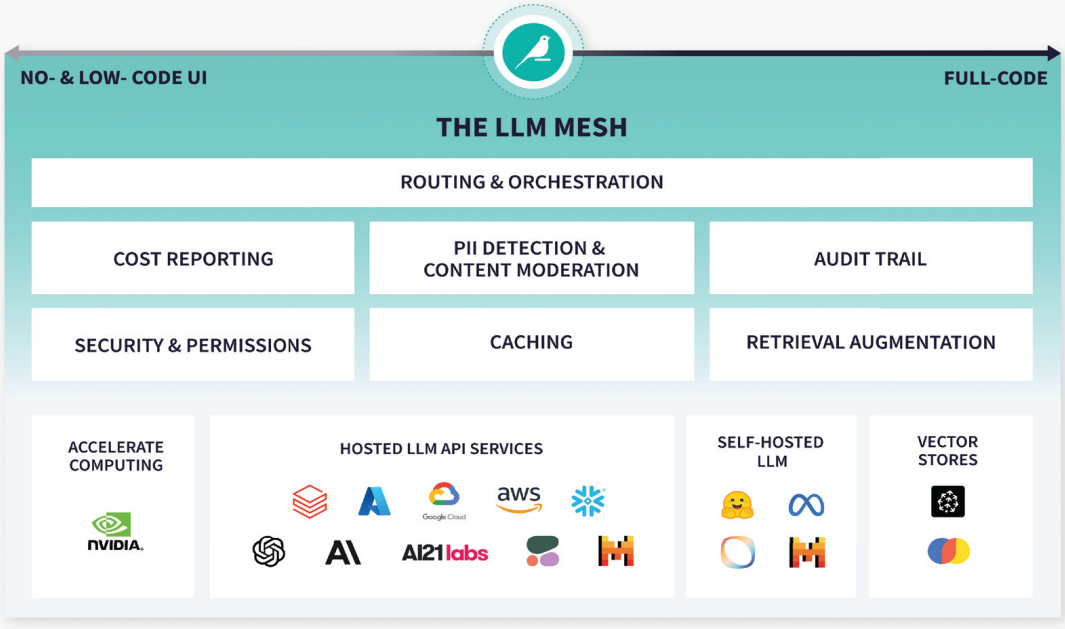
No-code text recipes enhanced with pre-trained Hugging Face models and LLMs for text summarization, classification, and other common language tasks.

PROMPT STUDIOS

Iteratively design and evaluate LLM prompts, compare performance and cost across models, and operationalize GenAI in your data projects.

GENAI SOLUTIONS

Pre-built Generative AI use cases and applications for even faster time to value.



LEARN MORE ABOUT THE LLM MESH

The LLM Mesh

A Practical Guide to Using Generative AI in the Enterprise

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Kurt Muehmel

Beijing • Boston • Farnham • Sebastopol • Tokyo

The LLM Mesh

by Kurt Muehmel

Copyright © 2025 O’Reilly Media, Inc. All rights reserved.

Printed in the United States of America.

Published by O’Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Editors: Jeff Bleiel and Aaron Black
- Production Editor: Kristen Brown
- Interior Designer: David Futato
- Cover Designer: Susan Brown
- Illustrator: Kate Dullea

- January 2025: First Edition

Revision History for the Early Release

- 2024-08-02: First Release

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *The LLM Mesh*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher’s views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O’Reilly and Dataiku. See our statement of editorial independence.

978-1-098-17661-7

[LSI]

Brief Table of Contents (*Not Yet Final*)

Chapter 1. Using LLMs in the Enterprise

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at *jbleiel@oreilly.com*.

"May you live in times of rapid technological progress." This is the blessing and the curse of our current moment. Recent advances in AI and a growing interest in technology, thanks to the release of wildly popular consumer products, have led to a frenzy of interest in, and use of, AI, and Large Language Models (LLMs) in particular, in the enterprise.

However, AI and LLMs remain nascent in the enterprise, meaning that best practices for their use are being defined. At the same time, the core technologies — the models themselves, technologies to host and serve the models, etc. — are evolving rapidly.

Table 1-1 provides a brief timeline of the release of various models and technologies that could be relevant for enterprise use. The diversity and speed of release create both opportunities and challenges when you are looking to use these technologies in production use cases.

Table 1-1. A (Non-Exhaustive) Timeline of Enterprise-Relevant Model and Product Releases

Developer or Provider	Model or Product	Release Date	Description
OpenAI	GPT-3	May 2020	175 billion parameter LLM with 2048 token context window
OpenAI	ChatGPT	November 2022	Consumer chatbot application, powered by GPT-3.5 Turbo
Microsoft Azure	OpenAI Service	January 2023	Managed service offering LLM from OpenAI

Developer or Provider	Model or Product	Release Date	Description
Amazon Web Services	Bedrock	September 2023	Managed service offering LLM from various developers
Dataiku	LLM Mesh	September 2023	Commercial LLM Mesh offering for connecting to LLMs and building LLM-powered applications in the enterprise

Developer or Provider	Model or Product	Release Date	Description
Databricks	DBRX	March 2024	Open-weights mixture of experts model with 132B total parameters and 32k-token input context window, licensed for commercial use

Developer or Provider	Model or Product	Release Date	Description
Meta	LLaMA 3 (8B, 70B)	April 2024	Updated LLM with 4096-token input context window, with updated license allowing certain commercial uses

Developer or Provider	Model or Product	Release Date	Description
Mistral	Mixtral 8x22B	April 2024	Open-weights mixture of experts model with up to 141B parameters and 64k-input context window, licensed for commercial use

Developer or Provider	Model or Product	Release Date	Description
OpenAI	GPT-4o	May 2024	Multimodal LLM supporting voice-to-voice generation and 128k-token input context window
Google	Gemini 1.5 Pro	May 2024	Multimodal LLM with 1M-token input context window

Today, you can build entirely new capabilities that would not have been possible previously, to improve the lives of your employees and better serve your customers. But you also have to keep up with rapid changes in the core technologies and use techniques that have not been fully proven. We are all now at the cutting edge.

This diversity of options among the technologies and techniques is truly a great thing. In fact, we are just scratching the surface for the potential uses of LLMs in the enterprise. It’s easy to imagine a future where these technologies are generating massive amounts of value for the enterprise, automating mundane tasks, and making new products and services possible.

In this chapter, we will briefly introduce what an LLM Mesh is, and then take an in-depth look at the many different types of LLMs that can be appropriate for use in the enterprise. We’ll discuss different characteristics of models, and how models are built, published, run, and perform.

After reading this chapter, you should be able to think about how you would want to use different models for different applications in your business. Given this multitude of models, you will see why an LLM Mesh architecture is going to be a key part of your AI strategy going forward.

What Is an LLM Mesh?

An LLM Mesh is an architecture paradigm for building LLM-powered applications in the enterprise. There are three principles regarding what an LLM Mesh should accomplish. An LLM Mesh should enable you to:

1. Access various LLM-related services through an abstraction layer.
2. Provide federated services for control and analysis.
3. Provide central discovery and documentation for LLM-related objects.

These principles allow for LLM-powered applications to be built in a modular manner, simplifying their development and maintenance.

Figure 1-1 illustrates an LLM Mesh architecture being used to develop two applications. Various objects, referenced in the Catalog and accessed via the Gateway, are combined to build the logic of the applications. Federated services provide control and analysis throughout the lifecycle of the application.

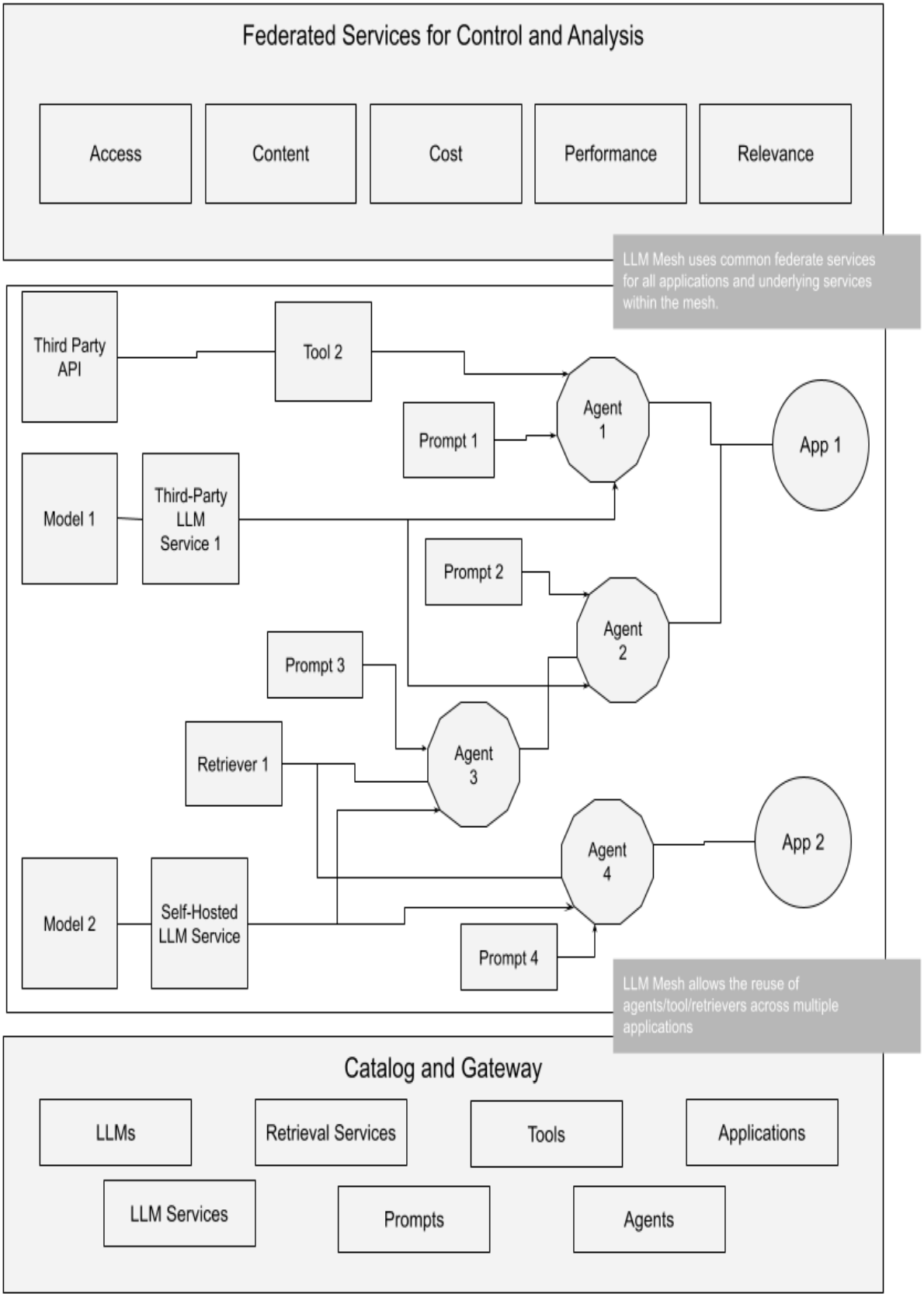

Federated Services for Control and Analysis
Access
Content
Cost
Performance
Relevance
LLM Mesh uses common federate services for all applications and underlying services within the mesh.
Third Party API
Tool 2
Prompt 1
Agent 1
App 1
Model 1
Third-Party LLM Service 1
Prompt 2
Agent 2
Prompt 3
Retriever 1
Agent 3
Agent 4
App 2
Model 2
Self-Hosted LLM Service
Prompt 4
LLM Mesh allows the reuse of agents/tool/retrievers across multiple applications
Catalog and Gateway
LLMs
Retrieval Services
Tools
Applications
LLM Services
Prompts
Agents

Figure 1-1. An LLM Mesh architecture

It is certainly possible to build LLM-powered applications in the enterprise without an LLM Mesh. Many of the initial applications that organizations have built since the release of ChatGPT do not use an LLM Mesh. In these cases, the logic for connecting the various objects of the application (LLM services, retrieval services, etc.) is built directly into the application, as are any additional capabilities such as access controls or logging. This approach is perfectly appropriate for building a first proof of concept, or a single application.

An LLM Mesh, however, becomes valuable when:

1. The total number of LLM-powered applications being developed begins to grow,
2. More teams start building and using the applications,
3. More complex LLM-powered applications are being designed and built.

In this context, the LLM Mesh will accelerate the development of the applications, simplify their maintenance, and help to ensure that the applications meet enterprise standards for safety, security, and performance.

WHY LLMS AND NOT GENERATIVE AI?

An LLM Mesh architecture focuses on LLMs and not Generative AI more broadly because LLMs are the core building blocks of the AI applications that will be built in the enterprise.

LLMs are large neural networks trained on text data. They possess a variety of natural language processing capabilities. Many, but not all, LLMs can generate text. Generative AI is a broader category of AI that includes models that can generate text, audio, images, and videos.

Beyond simply generating text, LLMs are also used to reason through a problem, to give instructions to various tools, and to write the code to connect to various tools. While image-generating models, for example, can be useful in the enterprise, they are not relevant in the context of building sophisticated AI applications that are the focus of the LLM Mesh.

An LLM Mesh provides a gateway not only to LLMs, but also to the full range of objects that are needed to build fully-featured, LLM-powered applications. These include the LLMs themselves and the services to host them, but also agents, tools, retriever services, and applications such as chatbots.

These objects are, for most organizations, new types of assets that will need to be developed and used. The skills to develop and use these kinds of objects are not yet commonplace in organizations, and best practices for their development and use are still being defined. Amid this rapid innovation, the LLM Mesh architecture paradigm aims to simplify the management and use of these objects to accelerate and standardize the development of LLM-powered applications. Chapter 2 will explore in depth

these different types of objects and how an LLM Mesh can simplify their use.

The Right Model for the Right Application

The challenge for the use of LLMs in the enterprise is not a lack of availability of models. As of June 2024, the popular model repository Hugging Face lists 727,354[1] models, of which 114,796[2] are text-generation models. More models are being developed and released every day.

In fact, the abundance can actually be a hindrance, as you have to sort through the many different options to choose the ones that are best for your applications.

A large general model that can do most things pretty well is a good place to start. But as an enterprise's use of LLMs matures and it seeks higher levels of performance and optimized budgets, it will need to use a growing number of models across different applications.

The following sections explore the different characteristics of models and how these characteristics may make a model more or less appropriate for the many different, specific uses in the enterprise.

Model Size: The Upside and Downside of More Parameters

The word “large” in large language model refers to the number of parameters in the model. Alternatively, “large” may refer to the number of tokens in the training data that the model is trained on. More training tokens lead to more parameters.

LLMs often have hundreds of billions to trillions of parameters. For example, GPT-3, released in May 2020[3] and the immediate precursor to the model behind the first version of ChatGPT, has 175 billion parameters. The first version of LLaMA from Meta AI in February 2023 has 65 billion parameters.[4] Increasingly, the makers of proprietary models are no longer making the number of parameters in their models public.

These parameters are the numerical values (sometimes they will be called weights and biases) that make up the simple mathematical formulae of each neuron in the neural network. Usually, they are 32-bit floating point numbers. A process called quantization can simplify these numbers to 4- or 8-bit integers. This process can often dramatically improve the efficiency of a model while having only a modest impact on model performance.

Figure 1-2 illustrates a simple neural network architecture, showing the input layer, two hidden layers, and the output layer. The circles represent the nodes in the network, the values under the nodes are the biases, while

the values on the lines connecting the nodes represent the weights. Larger neural networks, like LLMs, are built on the same basic architecture but are billions of times larger with more than one hundred hidden layers.

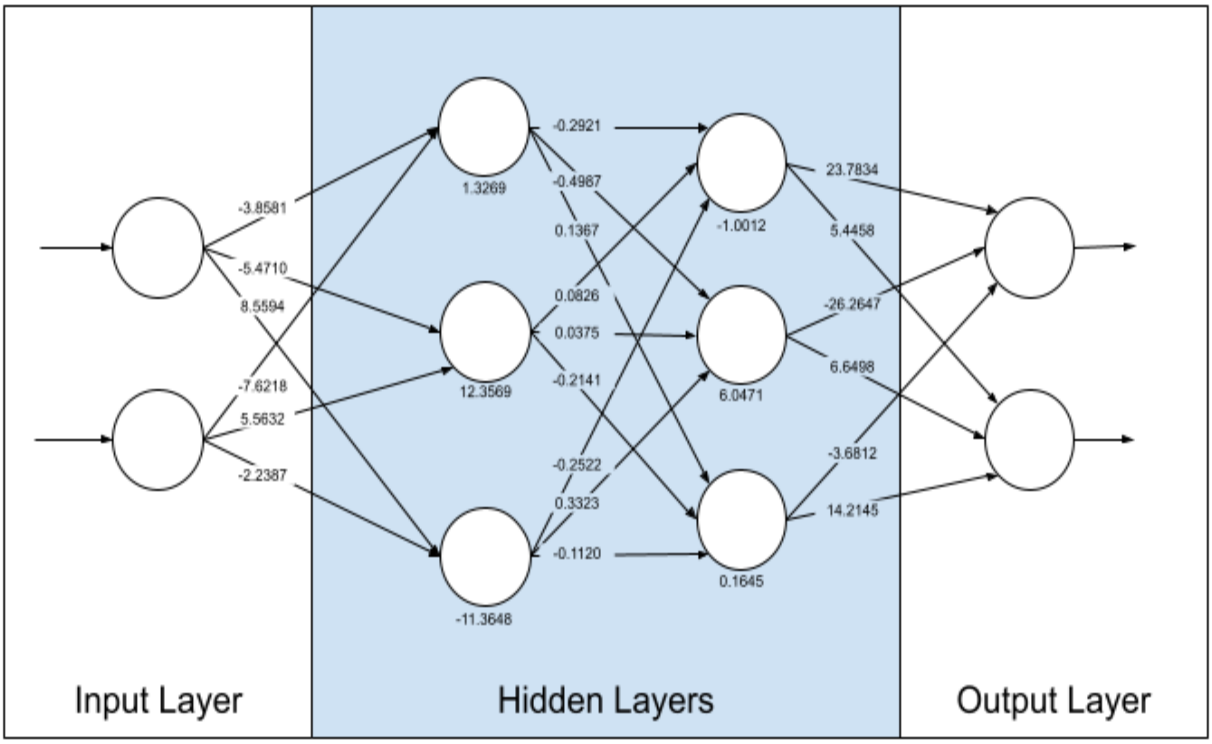


Figure 1-2. Simplified example of a neural network showing the input, hidden, and output layers and the weights connecting each node and the biases of each node

Generally speaking, larger models perform better: They can do more tasks and they can do those tasks better. Thus, it could be easy to conclude that you should choose the largest model your budget allows and use it for everything. But that would be like using your large, comfortable, powerful grand touring car for every trip. While it would be the right choice for a cross-country roadtrip, it would be overkill for a quick trip to the grocery

store or the bakery around the corner. A bicycle or your own two feet would be better for such errands.

The following subsections explain the tradeoffs related to the size of a model.

Inference Costs

The most direct impact of a larger model size will be on inference cost. Inference is the process of generating tokens in response to a particular input. A model with more parameters will require more calculations during inference. One way or another, those calculations must be run on some hardware that is installed and managed somewhere and that is consuming electricity, for which someone will have to pay the bill at the end of the month.

In some cases, companies offering these models as a service may obfuscate these costs, for example, by subsidizing the cost in order to gain more customers. This may make an apples-to-apples comparison difficult. We'll dig into cost considerations in Chapter 3.

Some models function, in essence, as a combination of smaller models, each specialized in different tasks. This architecture, known as Mixture of Experts (MoE), can dramatically reduce the cost of inference. One well-known model using an MoE architecture is 8x7B (also known as Mixtral) from Mistral. Despite being a 46.7-billion parameter model, only 12.9-

billion parameters are used per token. This approach has led to improvements in inference cost, but makes the model more challenging to build and to fine-tune.

All other things being equal, larger models will be more expensive to use, though technological advancements like MoE mean that these tradeoffs will become more complex in the future. The benefit that large models bring to a particular use case may justify their expense in certain cases, as discussed in the sections and chapters below, but a wise strategy will use them only where needed.

Inference Speed

While the inference of a larger model will require more calculations, these calculations can be done more quickly when using larger and higher-performance hardware. Furthermore, many of these calculations can be parallelized, using multiple processing units at the same time to run all of the necessary calculations. Again, MoE models do not need to use all parameters for every task.

Standard benchmarks are being established to accurately quantify and compare the speed of different models, acknowledging that hardware and network performance will have a significant impact on the results. The two metrics that are used most commonly are latency and throughput:

- *Latency*, often measured as Time To First Token (TTFT), is a measure of how long the model takes to generate its first response token to a user's input. In applications where the end user is interacting with the model in real time, latency will influence whether the model "feels" responsive. In applications where the model's response is part of a longer chain of interactions, latency will need to be considered when setting when the application will time out.
- *Throughput*, often measured as Tokens Per Second (TPS), measures the overall rate at which the model will generate tokens in response to a given request. Like latency, it will influence if a model feels fast to an end user. Throughput needs to be taken into consideration when building applications that depend on the output of the model.

When comparing the speed of models, pay close attention to the units being used, as different testers are using different methodologies.

While the relationship between model size and inference speed is indirect (because large models can be run more quickly on higher-performance hardware, and factors like network performance can influence the time it takes to receive a response), the measured speed of a deployed model must be taken into account when building an LLM-powered application.

Task Coverage and Performance

One of the main functional differences between LLMs and previous generations of models used for Natural Language Processing (NLP) is that those earlier models were always task specific. For example, separate models would be used for sentiment analysis, text summarization, or language translation.

LLMs can do all of those tasks, and generative LLMs can do something that previous models could not: Generate text based on a prompt. Generally speaking, models with more parameters can perform more tasks, which can be useful in the enterprise when several tasks need to be performed on the input text.

LLMs have also been shown to gain new abilities as they grow larger and are trained on more and more data. These emergent abilities are unpredictable. In other words, researchers cannot predict ahead of time at what point in its training a model will gain new abilities.[5] It is possible that future LLMs will be capable of many more tasks or will see dramatic improvement in their performance of existing tasks as they grow larger.

In addition to gaining new abilities as they grow, larger LLMs generally show better performance on any task that they are capable of performing as well. Recent research shows that this improvement in performance is not linear nor predictable, as with the emergent abilities mentioned above.[6]

Context Windows

The amount of input text that a model can receive within a single prompt is known as its context window. Measured in tokens, it defines how much information a model can work with at a single time.

For example, a model with a small context window can only be used to summarize a document that can fit in its context window. You could break up the document into smaller pieces, but the model would summarize each separately, without knowledge of the entire document, potentially resulting in repetitive or incoherent results. Large context windows, on the other hand, allow for plenty of space to provide examples of what you want the LLM to produce (called few-shot learning) and to engage in more complex prompt engineering techniques.

Generally speaking, larger models have larger context windows, and some models have been optimized for exceptionally large context windows. While the original GPT model had a context window of only 512 tokens (approximately one page of text) Gemini 1.5 from Google now has a context window of more than 1 million tokens and has been shown in internal testing to handle up to 10 million tokens.

Sizing Models to the Task

Reading these previous sections, it is easy to conclude that if cost and complexity are no barrier, then the largest models are always the best choice for any application in the enterprise. But, in which enterprise are cost and

complexity not a barrier? In fact, these are the two greatest barriers to the practical use of LLMs in the enterprise!

Given this reality, enterprise users of LLMs will need to choose a model that strikes the right balance of ability, performance, cost, and complexity for a specific application. The right choice for one application may not be the right choice for another application.

General Models vs. Specialized Models

Building on this understanding of the implications of model size, we will now explore the differences between general models and specialized models.

General models are those that have been trained to perform at human level across a wide range of tasks. OpenAI's GPT-4 (released in March 2023[7]) is an excellent example of such a model. It demonstrates very high performance across a great number of tasks, covering natural languages, programming languages, and a wide variety of specialized jargon. It can generate, summarize, and translate text, it can write technical reports, and it can write poetry. Furthermore, GPT-4 can take image data as input, a capability known as multimodality.

In contrast to these general models, specialized models have been trained to perform well on specific tasks, in specific domains, or have been

compressed and optimized for performance at a smaller model size.

Note that while high-performing, general models tend to be larger models, specialized models may be larger or smaller.

Types of Specialized Models

Task-specific models are those that are focused on doing specific tasks very well. Some examples of task-specific models include M2M100[8], a model that is designed to translate between any pair of natural languages, or OpenAI's Codex[9], an evolution of GPT-3 that is trained specifically to generate code across a wide variety of programming languages. A common application might be a model that is specialized in summarization, allowing it to be much smaller than a general model. Thanks to its small size, it could run locally on a mobile device and be used for rapidly summarizing content directly on the phone.

Domain-specific models are those that are trained on the language of a specific domain. For example, BioMedLM[10] is a 2.7-billion parameter model trained on biomedical literature and is thus well-adapted to answering questions about medical topics, while BloombergGPT[11] is a 50-billion parameter model trained on a very large dataset of financial documents designed to serve the financial services industry.

Resource-constrained models are models that have been compressed through various techniques to maintain good performance in their desired

tasks or across a wide range of tasks, while being less resource intensive to run. An example is MobileBERT[12], a compressed version of the popular BERT model designed to be run on mobile devices.

Embedding models transform text into numerical representations called embeddings or vectors. These embeddings capture the semantic meanings of the text and the relationships between the different parts of the text. A common application is retrieval augmented generation (RAG) where a corpus of text (e.g., thousands of documents) are converted into embeddings and stored in a specialized database called a vector store.

Reranking models are used to refine the initial ranking of search results of an embedding model to make the results more relevant to the end user. There are LLM-based and non-LLM rerankers, each presenting tradeoffs in terms of performance and quality of response.

Choosing a General or Specialized Model

The existence of a diverse and growing ecosystem of both general and specialized models gives enterprises the opportunity to use different models for different purposes.

In the enterprise, general models are well-suited to tasks where the input is going to be highly unpredictable. This could be the classification of documents into different categories. For example, if a directory contained a mix of contracts, invoices, and emails, a first step in the analysis could be to

use a general model to sort the documents into different categories so that the contracts could be analyzed separately from the invoices.

Specialized models are well adapted for tasks where the input data is more homogeneous and predictable. Let's explore what this might look like across a pharmaceutical company. That company may wish to build a chatbot to serve its customers (doctors, nurses, pharmacists, and other healthcare providers) in their interactions with patients. It would likely choose a domain-specific model like BioMedLM to ensure higher quality and more relevant results. The same company may then use a model like ESM[13] from Meta AI researchers which has been trained on the language of proteins as part of their molecular research applications. Finally, that same organization may use a non-LLM computer vision model to watch their products as they come off of the manufacturing line to quickly identify any anomalies as part of their quality assurance processes.

General models can be a very good starting point for enterprises as they experiment and build their first use cases using LLMs. At those early stages, the simplicity of using a single model for a variety of tasks and use cases outweighs the benefits of further optimization using specialized models. But, as an enterprise scales its use of LLMs across use cases, enterprises will want to optimize their use to improve performance and reduce costs. In this context, specialized models become more relevant, and the number of models that an organization will need to manage and apply will tend to increase.

WHAT IS FINE-TUNING?

A common way of creating a domain-specific model is to fine-tune an existing base model. For example, this is how BloombergGPT was built, by fine-tuning the open-access BLOOM model on a proprietary dataset of financial documents.[14]

Fine-tuning is a type of transfer learning that feeds new — usually specialized — data into a model to retrain some parts of the model on this new data. Compared to building a model from scratch, it is far less complex and compute-intensive.

While fine-tuning is simpler and less expensive than building a base model, it remains an advanced technique and should be used only when other, simpler, and less expensive avenues have been exhausted.

Fine-tuning has often been cited as a way to elicit better performance from base models, allowing enterprises to differentiate their use of LLMs from their competition. While this is true, it ignores or downplays the difficulties of fine-tuning and leaves unexplored the opportunity to generate differentiated results using simpler techniques like prompt engineering and Retrieval Augmented Generation (RAG).

Making Sense of Model Licenses

There has often been a conflation between a model's license (e.g., open source vs. proprietary) and where the model is hosted (e.g., provided as a service via API vs. self-hosted or on-premises). It is important to distinguish between the two dimensions. For example, hosted services like Amazon Bedrock serve both proprietary and open models, while providers like Cohere license their proprietary models for self-hosting in addition to hosting the model themselves. Hosting options will be covered in the next section, while this section will distinguish between the different license types.

Proprietary Models

Proprietary models are just that: proprietary to their creators. The creator of a proprietary model retains full control over the intellectual property of the model itself. Most often, these models are a black box. In other words, their training data, the algorithm used to train the model, any subsequent steps such as reinforcement learning or fine-tuning, and the weights of the model itself remain hidden from the end user, unless the developer chooses to disclose any of this information.

Early in the development of LLMs, there was a trend towards openness, even among developers of proprietary models. OpenAI published a technical paper detailing the development process of GPT-3.[15] The release of subsequent models, such as GPT-4, have not been accompanied by such detail.

The use of proprietary models is governed by the terms of use that a customer agrees to when using the model. An enterprise should ensure a full and detailed legal review of these terms to ensure that they are appropriate for the intended use. Specific attention should be given to any rights that the model provider may claim to have on any data sent to the model for inference. Generally, models that are licensed for professional use do not retain any customer data for retraining purposes, though they may retain customer data for quality assurance purposes.

Open-Weights Models

An open-weights model provides public access to the pre-trained parameters of the model. This can allow the end user to modify the weights through fine-tuning or other techniques to adapt the model to their needs.

Open-weights models typically do not publish their training data, training algorithms, or other associated information. As such, it can limit the ability to perform a detailed technical inspection of the model or to reproduce the model’s performance. These limitations, however, are most relevant to other researchers and are less relevant to enterprises that are seeking to simply use a model in the most efficient and effective way.

Open Access Models

Open access is a growing category of models that are nearly open, but have custom terms that cannot be considered fully open source in the traditional definition of that term. It covers a wide gamut of licenses with different restrictions, and thus should be the subject of a detailed legal review to ensure that the license allows for the intended use.

Some examples include:

- BLOOM, which was released under the OpenRAIL-M license.[16] Though quite nearly open source, it has requirements for responsible use of the model, which means that it is not fully open source.
- LLaMA 2 and 3 from Meta AI, which have been released with their own custom licenses (called the LLaMA 2[17] and LLaMa 3[18] Community Licenses, respectively) that set limits to the use of the model. Specifically, the licenses forbid the use of the model in applications with more than 700 million monthly active users and for the purpose of building competitive models.

Open Source Models

Open source models are the most open of all, publishing details of their training data, training algorithms, and model parameters, allowing for the most permissive use of the model. Common open source licenses include Apache 2.0 and MIT. Meta AI's first version of their LLaMA model was

released under a GPLv3 license, restricting it from commercial use and thus making it not useful for most enterprise applications.[19]

Choosing a License for Enterprise Use

Even though proprietary models are the most restrictive, they are often entirely appropriate for use in the enterprise, as with any other proprietary enterprise software. By charging for access to their models, providers of proprietary models may be able to more easily provide for services and support for the use of their model. This may make their use more appropriate for use in the enterprise.

Open-weights, open access, and open source models may be more useful in applications where an enterprise wants more control over the model itself and possesses the technical expertise to make any such modifications or to host the model.

Model Hosting

Enterprises have three main hosting options when looking to access LLMs:

1. API services from the model developers, such as OpenAI, Anthropic, Cohere, and Mistral.
2. Cloud Service Providers offering hosted LLM services, such as Azure OpenAI Service, AWS Bedrock, or Google Vertex AI Model Garden.

These services also allow customers to load their own models, while the underlying hardware is managed by the cloud provider.

3. Self-managed hosting of models. Many models with different licensing terms are available for self hosting, including open-source and open-access models as described above. Cohere also licenses its proprietary models for self-managed hosting.

In many cases, models hosted by their developer or a cloud service provider (options 1 and 2, above) are the best choice in the enterprise. In the same way that cloud computing outsourced the burden of running data centers, hosted models are a simple continuation of that trend, offering infrastructure-as-a-service. Given the intensive compute requirements of LLMs, especially under heavy workloads, outsourcing this can be a wise choice.

The most common objection to using a hosted service is that it requires sending corporate data to a third-party service. But, in many cases, this corporate data is already hosted by a third party that may also be hosting internal communications and other sensitive data (e.g., a company that uses Microsoft 365 productivity and communication tools has its data in Azure). Is using the LLM service from that same provider any different? It is ultimately a question that warrants review by your legal and risk teams, but in most cases, the conclusion is that it is not different in a meaningful way.

Self-hosting a model requires acquiring the necessary hardware, configuring it to run the LLM, and then maintaining that stack for reliable internal use. Typically, this will require a cluster of GPUs that have been properly configured with the right drivers and packages to run the LLM in question. The LLM must then be loaded into this environment so that it can begin to serve internal requests.

Self-hosting can be an appropriate choice for an enterprise in cases where an organization needs full control over the model and the hardware it runs on and cannot use a third-party service for its data. This may be the case in the most restrictive data environments, or if the enterprise does not want to rely on a third party to ensure the performance of the environment, notably in contexts where third-party providers may need to throttle access to certain customers to ensure the overall stability and availability of their service.

In the case of both self-hosting and hosted services, applications that use the LLM will access the model through an API endpoint. The difference is simply who is hosting and maintaining that endpoint and whether the data going to and returning from the LLM leaves the corporate firewall of the enterprise.

Building a Base Model Is Not for Most Organizations

Early on in the popular interest in LLMs, a lot of attention was given to the expense and complexity of building these models. Billions of dollars were being spent building these models, and sometimes training them took many months. A huge amount of this initial work was amassing the enormous training sets required to build models of this scale.

Recent advances have brought down the time needed to build new models, and open source training data repositories now exist. But the fundamental question for an enterprise that is considering building a model remains: Why would you? Given the great diversity of today's models, which offer seemingly endless combinations of performance, specificity, licensing, and hosting options, what would justify the time and expense needed to build your own model, especially given that you are uncertain of being successful?

Any company whose core business is not building or serving AI models should not consider building their own model. There are more than enough options on the market today. The challenge is not getting access to a model, but using it safely, securely, efficiently, and effectively to further your business goals. This is where an LLM Mesh comes into play.

Bottom Line: Why the LLM Mesh?

As you have read in the previous sections, a great variety of models exists in an ecosystem that is rapidly evolving. This is ultimately a very good thing for enterprises: It means that they will be able to pick and choose the right model for the right applications within their business. Building applications that are powered by these LLMs requires combining them with other objects, like retrieval systems, prompts, and tools. This requires careful attention to many different factors:

- How the models, services, and associated objects are registered and used within the organization,
- How the data is routed to the model,
- How access to the models and services is controlled,
- How the use of the model is logged and audited,
- How the content generated by the model is moderated,
- How the models can be enriched with proprietary data,
- How can the applications be developed, deployed and maintained efficiently, and
- How can more people become involved in this process?

As more LLM-powered applications are built and used in the enterprise, the cost and complexity of managing all of these dimensions risks spiraling out of control. This could force the enterprise to make compromises, potentially limiting the value that it derives from AI.

For example, perhaps there is a use case that would benefit from using a small, specialized model that is self-hosted and to which access is restricted. This could be a code assistant that is well-versed in the company's proprietary code libraries. If the organization lacks the ability to quickly and efficiently add this model to its mix, it may not pursue this use case, leaving the potential gains in efficiency on the table and falling behind its competition.

This would be unfortunate, given that many of the additional capabilities that are required to use an LLM efficiently and effectively in an enterprise are common to all models.

This is the power of an LLM Mesh: its ability to reduce the cost of building an additional LLM-powered application in the enterprise. With an LLM Mesh, an enterprise is free to develop an optimal AI strategy without compromising on performance, cost, safety, or security.

The remaining chapters of this technical guide will go into much more detail about how implementing an LLM Mesh can be done.

1 https://huggingface.co/models, accessed June 20, 2024

2 https://huggingface.co/models?pipeline_tag=text-generation&sort=trending, accessed June 20, 2024

3 https://arxiv.org/abs/2005.14165

4 https://ai.meta.com/blog/large-language-model-llama-meta-ai/

5 https://openreview.net/pdf?id=yzkSU5zdwD

6 https://arxiv.org/abs/2210.14891

7 https://openai.com/index/gpt-4-research/

8 https://about.fb.com/news/2020/10/first-multilingual-machine-translation-model/

9 https://openai.com/index/openai-codex/

10 https://arxiv.org/abs/2403.18421

11 https://arxiv.org/abs/2303.17564

12 https://arxiv.org/abs/2004.02984

13 https://github.com/facebookresearch/esm

14 https://arxiv.org/abs/2303.17564

15 https://arxiv.org/abs/2005.14165

16 https://bigscience.huggingface.co/blog/the-bigscience-rail-license

17 https://llama.meta.com/llama2/license/

18 https://llama.meta.com/llama3/license/

19 https://arxiv.org/abs/2302.13971

About the Author

As Everyday AI Strategic Advisor at Dataiku, **Kurt Muehmel** brings Dataiku's vision of Everyday AI to industry analysts and media worldwide. He advises Dataiku's C-Suite on market and technology trends, ensuring that they maintain their position as pioneers. Kurt is a creative and analytical executive with 15+ years of experience and foundational expertise in the Enterprise AI space and, more broadly, B2B SaaS go-to-market strategy and tactics. He's focused on building a future where the most powerful technologies serve the needs of people and businesses.